

Nupano Platform Services QA

Overview

Nupano is a cloud native platform that enables digital business model on machines. It has two parts

- **Engineering – everything that runs in cloud**
- **Runtime – everything that is outside the cloud**

This page provide information related to QA activities and design test automation for cloud-based components known as Platform Services (web-based and restful web services).

The product requirements are in the form of user stories for which test cases are written.

Since we are not using any Test Management Tool, the test cases are designed and maintained in plain English text (called Gherkin with Given-When-Then format) on Confluence (a collaborative online documentation tool).

Manual and test automation is performed on the stories implemented by the developer. In NUPANO (agile process) development, the builds happen frequently. Test automation is triggered for stories in previous builds that are integrated with developer builds (these builds are released for testing and are the replica of the production build). Manual test activities are done for story(s) (not automated or automation development in-progress) on the current developer build.

Test automation can be triggered in following ways.

- **Manually, where tester/ developer can manually trigger the test**
- **Scheduler, where automatic trigger happens at scheduled time**

To achieve test automation in Nupano, test automation framework is developed. It is a hybrid framework as it supports multiple frameworks and libraries. The tests are designed using [Selenium](#) (open-source java library used for browser automation) and Core Java. The test reports generated are in HTML format.

Test scripts and test automation framework code is maintained in the repository at GitHub (a cloud-based code hosting platform for version control and collaboration). Build tool, Gradle is used for build management and test execution engine.

The test environment is docker based (virtual software platform that simplifies the process of building, running, managing, and distributing applications) and has no dependency on any specific system to execute the test.

Github contains a tool, GitHub actions (cloud based continuous integration and continuous delivery platform that allows to automate a build, test, and deploy) in which workflow (series of actions to be performed in sequential way) is designed to automate the test process.

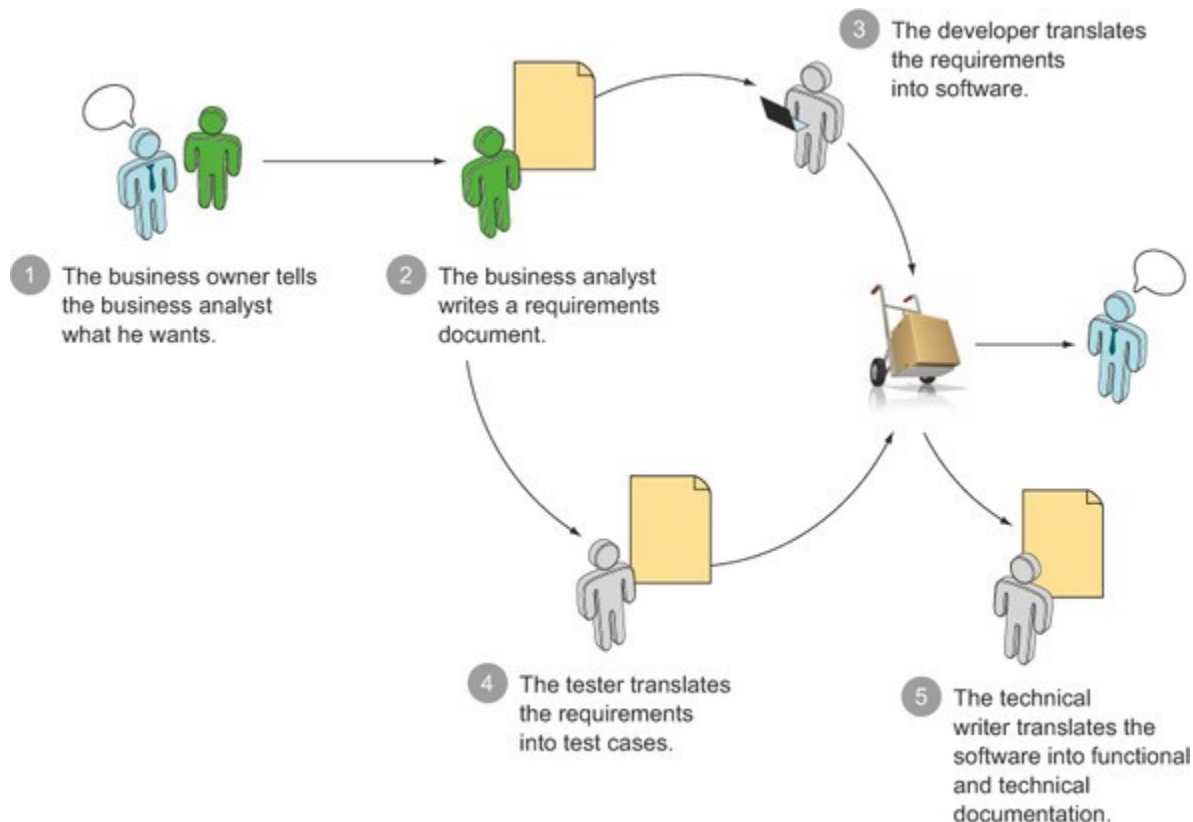
Github action performs the following actions sequentially:

- Pulls the latest docker image
- Triggers the functional test
- Generate the test artifacts
- Copies them to AWS (cloud-based storage)
- Publish the test data for Dashboard

The [dashboard](#) data is accessible for team members for further analysis. Additionally, test summary with link to test report are logged in Jira (project management and issue tracking software).

Test automation helps in reducing the time for execution of regression test. At present (for NAT), test automation is done for 53 user stories that include 328 functional and 39 restful web services test cases. The execution time of regression test for functional scenario is 1 hour (vs manual duration 1 week) and restful web services is 5 min (vs. manual duration 1 day).

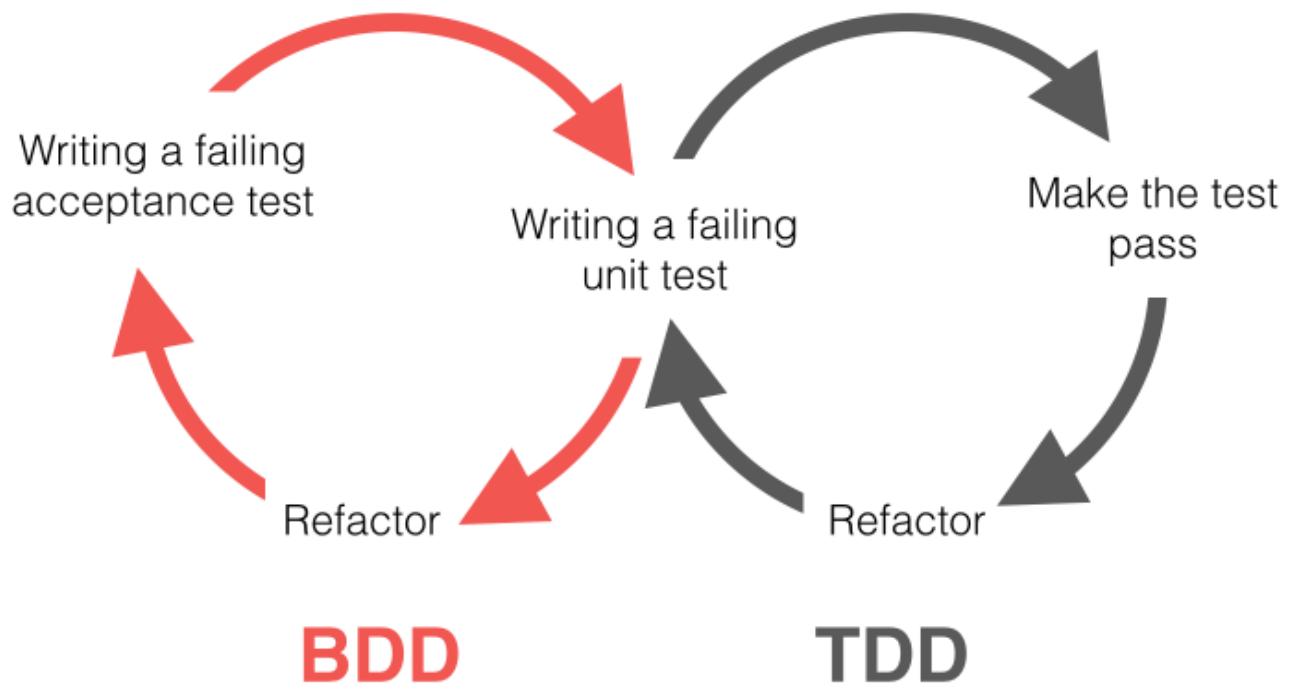
Testing in Agile through Behaviour Driven Development (BDD)



Agile methods avoid the inefficiencies and overheads in traditional testing approaches by introducing the concepts like test automation, collaborative testing, full life cycle testing, reduced overheads, and minimum documentation.

Behavior Driven Development (BDD) — Tests are written in a non-technical language that everyone can understand (e.g. a domain-specific language like Gherkin). BDD combines the principles of Test Driven Development (TDD) and Acceptance Test Driven

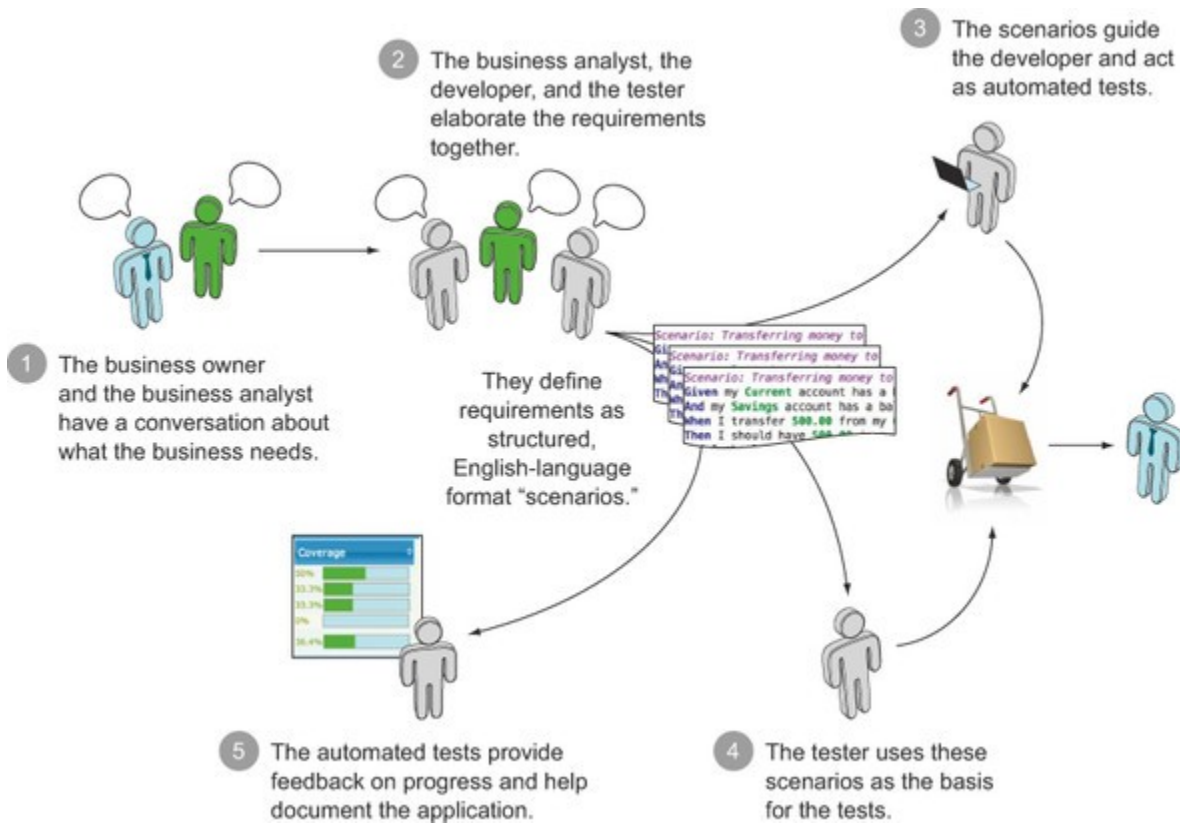
Development (ATDD) and forms an approach for building a **shared understanding** on what kind of software to build **by discussing examples**.



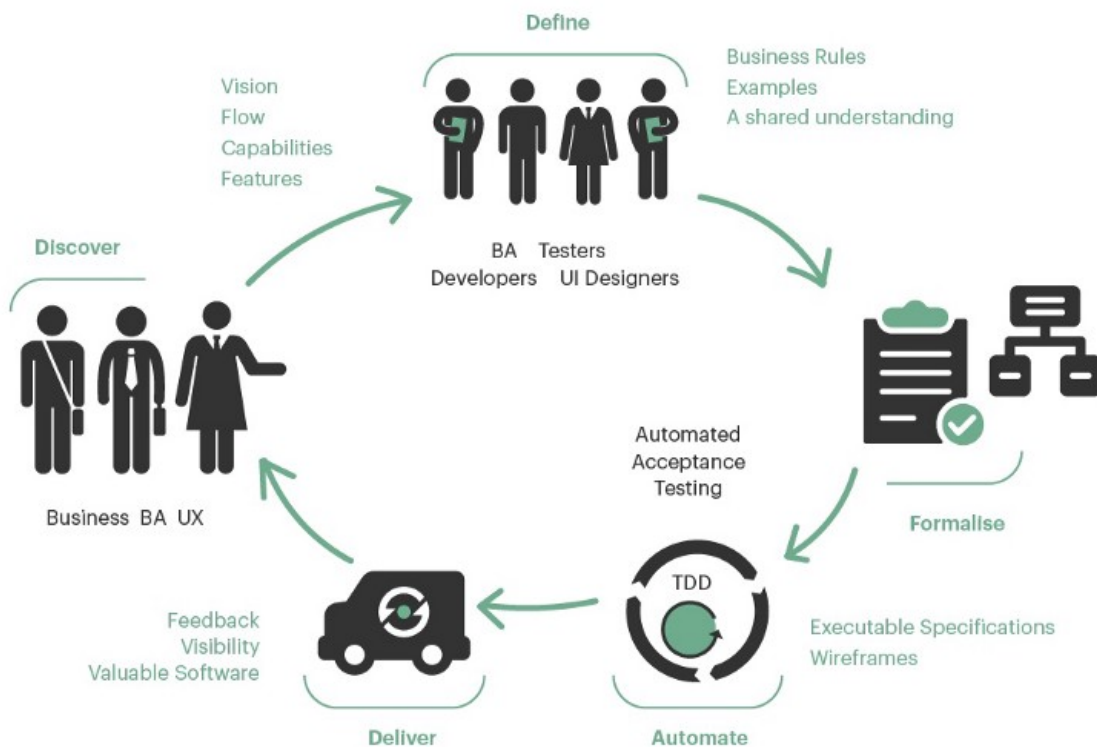
In Agile environments, BDD plays a vital role because it strongly encourages the use of Agile methodologies during the development and testing. BDD brings customers, end-users, BAs, QAs, and SEs of the software product into one table for effective sharing of knowledge on the system and its testing requirements.

BDD consists of cycles of a set of steps to follow.

1. Identify business **feature**.
2. Identify **scenarios** under the selected feature.
3. Define **steps** for each scenario.
4. **Run** feature and **fail**.
5. Write code to **make steps pass**.
6. **Refactor** code, Create **reusable automation library**.
7. Run feature and **pass**.
8. Generate **test reports**.



BDD involves in a cycle of above activities and requires the involvement and collaboration of a cross-functional team which consists of customer/client, BAs, QAs, Testers, UI & UX designers, and software developers. It is a method carefully developed while keeping the principles of Agile Manifesto in mind.



The biggest advantage of behavioral tests might be that they describe a set of functions that a user expects from a system in a very concrete and direct manner. The sum of these behaviors essentially document a contract with the user/client. If any of the tests fail, this contract is not upheld.

Behavioral tests can be written for a system at any time: before, during, or after development. The case where the tests are written first is termed behavior-driven development.

The suite of unit tests provides constant feedback that each component is still working. The unit tests act as documentation that cannot go out-of-date, unlike separate documentation, which can and frequently does. When the test passes and the production code is refactored to remove duplication, it is clear that the code is finished, and the developer can move on to a new test.

Test-driven development forces critical analysis and design because the developer cannot create the production code without truly understanding what the desired result should be and how to test it. The software tends to be better designed, that is, loosely coupled and easily maintainable, because the developer is free to make design decisions and refactor at any time with confidence that the software is still working. This confidence is gained by running the tests. The need for a design pattern may emerge, and the code can be changed at that time.

The test suites act as a regression safety net on bugs: If a bug is found, the developer should create a test to reveal the bug and then modify the production code so that the bug

goes away and all other tests still pass. On each successive test run, all previous bug fixes are verified. It also reduces debugging time.

NUPANO Test Automation Overview

NUPANO Test Automation Framework is developed to support the execution of weekly regression tests, scenario-based tests, and user story verification. This framework helps in testing web-based application and restful web services.

The framework is a hybrid model, and its core design is based on [Serenity BDD](#) framework with core JAVA used as programming language. The framework design and architecture support CI/CD by default.

At present, NUPANO Test Automation Framework includes & executes test cases for 53 user-stories. The user-stories includes 328 functional and 39 restful webservice test cases.

The tests are designed using [Cucumber BDD](#) framework in plain English text called as Gherkin and is the entry point of the framework. The framework supports both Gradle and Maven as build management tool and test execution engine.

The test cases are further isolated through Junit annotations and [Cucumber Tags](#) which helps framework to execute a certain test case or group of test cases. As the framework supports CI/CD, a workflow has been designed in [GitHub Actions to trigger the scenarios both manually, automatically and via scheduler](#).

NUPANO Test Automation Framework also consumes a [Webhook](#) created in GitHub Actions which enables and extends the execution of these tests in other GitHub action workflow and process. This feature helps to directly integrate and execute the test inside the NAT Test Automation Framework with any development workflow.

The test environment for NUPANO Test Automation Framework is docker based and have no dependency on any specific system to execute the tests. The GitHub actions workflow automatically pull the latest docker image and then triggers the test execution.

The entire source code and test cases are stored on cloud-based version of GitHub hence, making it accessible from anywhere.

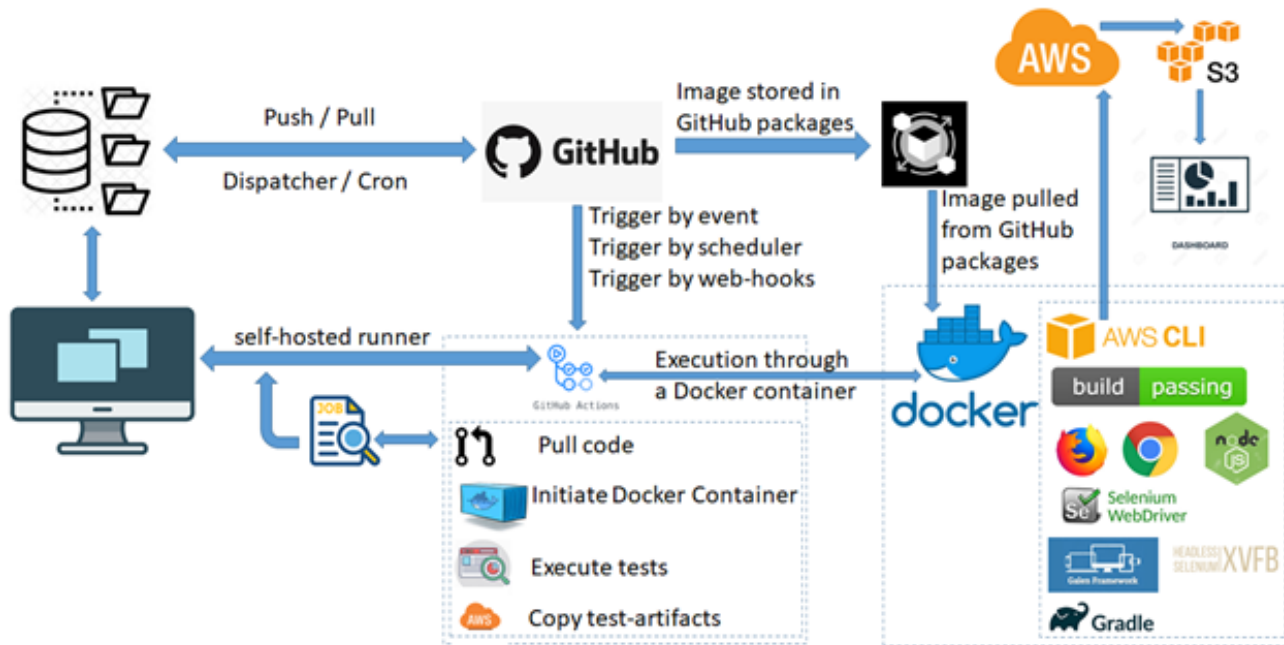
Regression tests are scheduled for every Saturday evening and get triggered automatically through GitHub actions workflow. With the current list of tests available, to execute the regression tests for functional scenarios execution time is 1 hour and for restful web services execution time is 5 minutes.

Verification of user-stories is triggered through AWS Infrastructure Manager (dashboard which lists all the User Stories in development) through [Webhook](#). For user story verification same set of tests get executed which were triggered during regression testing.

Scenario based testing can be triggered manually from the GitHub web page and needs to provide a cucumber tag of a specific test or group of tests.

NUPANO Test Automation Framework maintains and stores all the test artifacts on cloud storage provide by AWS. There is an [online dashboard](#) through which all the test artifacts are accessible. Additionally, when verification of user-story completes a test summary along with the links of the test reports are logged on JIRA against the ID of user-story verified.

NUPANO Test Automation Framework - Platform Services



The above model explains the test automation architecture designed for Platform Services.

QA team maintains the test automation framework code and test scripts in repository maintained in GitHub.

GitHub also stores a test infrastructure docker image under GitHub package.

The docker image contains the required test environment (browsers, frameworks, libraries, and build & test engine) to launch and perform automated testing.

The repository contains a workflow which is called by GitHub to trigger the set of actions defined under the workflow.

In case of Platform Services once the workflow is triggered:

- The latest code is triggered
- Docker Container is Initialized
- Tests are executed
- Test Artifacts generated during execution are copied to AWS

A test system needs to associate with a workflow which can be either a Virtual system hosted at GitHub / AWS or it can be a local hosted system connected with GitHub through Self-Hosted Runner.

Self-Hosted Runner is a GitHub Service which needs to be configured on the system required to be connected with GitHub. When the service is running on the system GitHub can identify the system and can perform the execution of the workflow on it.



The above model explains the workflow of test automation when QA team develops and integrates the automated test-scripts with the framework.

Test Automation can be triggered by 3 ways:

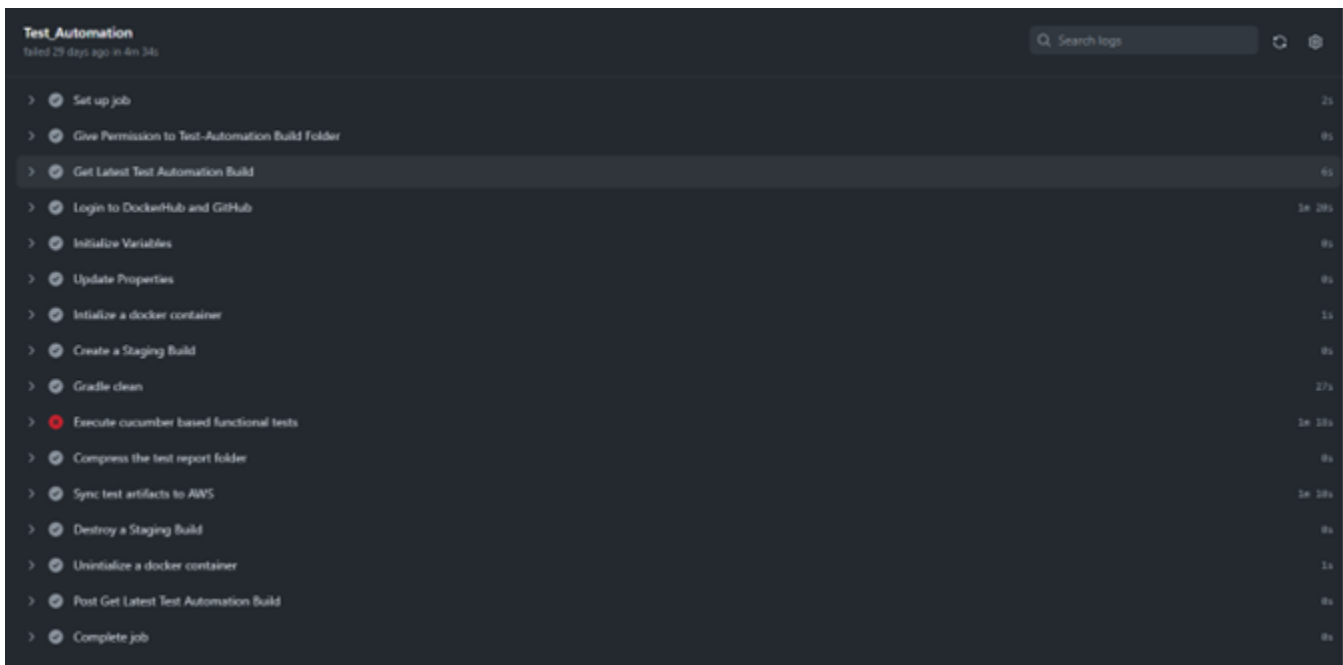
- **Manually:** This is an on-demand approach where any member from QA / Developer can manually trigger the test-automation by clicking the <Run workflow> from the GitHub Actions.
- **Scheduler:** This is a scheduled approach where workflow will get triggered automatically at a scheduled day and time.
- **Webhooks:** This is also an on-demand approach but more in lines with Continuous Testing and Integration approach. Webhooks are customizable GitHub action rest API which user can integrate with its workflow to trigger an external workflow. This means a developer can integrate a Webhook in its workflow process. Once the code is developed, compiled, build and successfully deployed, the development workflow can trigger the test-automation workflow. If the tests are successful, then code will get merged with Master branch else a defect will get raised and merging of code in Master branch will not happen.

Developer develops the code and merge it in the repository hosted at GitHub. This code gets compiled, build, and deployed in AWS.

QA team develops the test-scripts and merge it in the repository hosted at GitHub. Same repository also maintains the Test Automation Framework code.

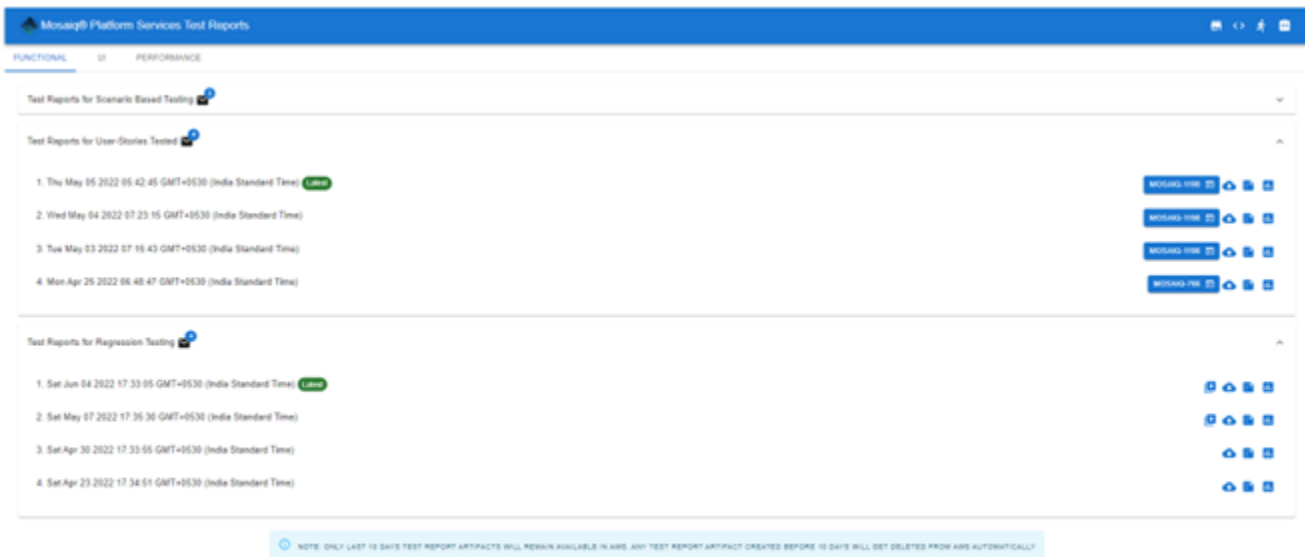
GitHub contains a tool known with a name GitHub action. This tool is used to automate the build, deploy, and test processes. To automate a process through GitHub action, GitHub repository should be configured with a Workflow. Workflow defines Step and every Step is associated with some desired action.

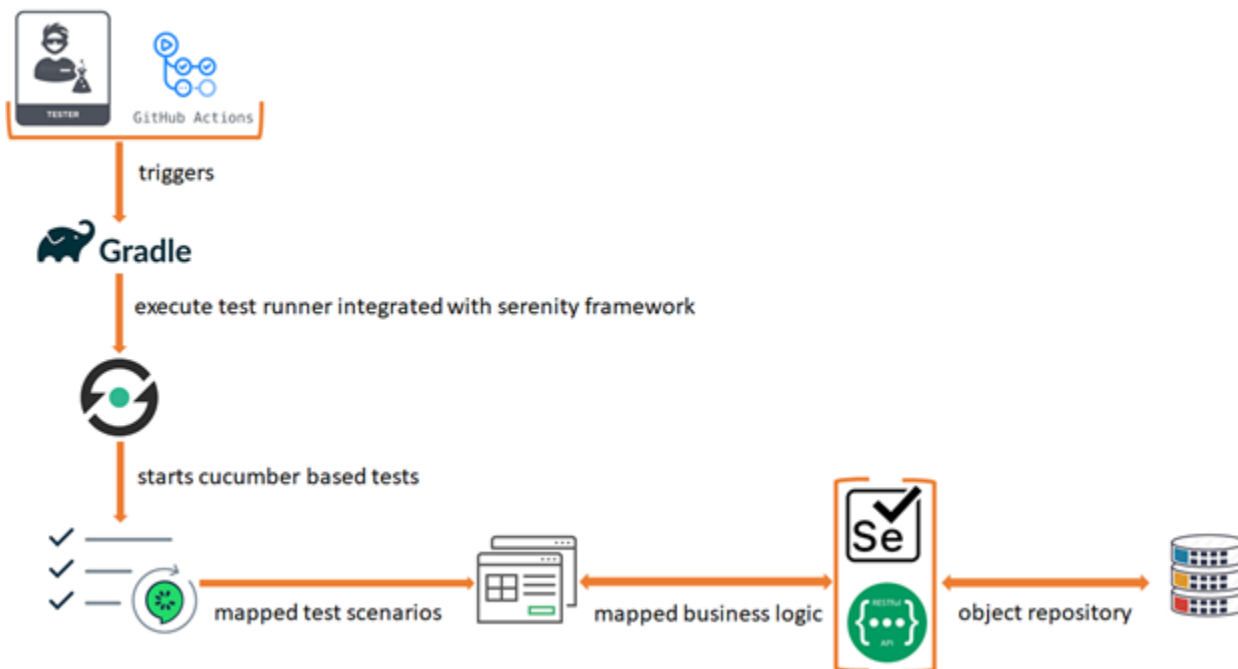
A screenshot of an executed GitHub action workflow for reference has been provided. This will show you the execution flow of Steps with desired Actions designed inside the workflow.



GitHub Action triggers the functional tests -> Generates the Test Artifacts -> Copies the Test Artifacts to AWS -> Publish the Test Data for Dashboard

A screenshot of Dashboard has been provided for reference. The Dashboard publish test data for all the NAT projects which also includes Platform Services.





The above model explains the execution flow of functional automated test-scripts once triggered by Github Action.

Gradle is a test engine which triggers the main / test runner class from the framework. The runner class will initialize the Cucumber based feature file. Each feature file will be mapped with its relevant java class which maps defined test steps in cucumber with its relevant java function. Every mapped java class will refer business logic and object repositories.

The mapped java classes are not tightly coupled with each feature file. In fact, these mapped java classes are loosely coupled and can get mapped with multiple feature files hence, making it flexible and shareable.

Same execution model is applicable to perform the frontend (functional) and rest API testing. Such approach gives QA team flexibility to either develop test scripts only for frontend requirements (functional) or rest API using the same framework. Additionally, QA team can develop test scripts which can combine frontend requirements (functional) and rest API.

The test report generated will combine the results of both frontend (functional) and rest API.

NUPANO Test Automation Framework - Tools & Technology

QA Team of NUPANO Platform Services uses latest technologies and framework to design and test the product. To automate different Test Automation types, QA team have identified various tools and frameworks which is highly ranked and valued as per current market trends.

Selenium

It is an open-source umbrella project for a range of tools and libraries aimed at supporting browser automation.

Rest Assured

It is a java library used for testing and validating the Restful Web Services. It supports Behavior-Driven Development (BDD) syntax like Given, When, and Then notations. It helps us to integrate with testing frameworks like Junit or TestNG.

Serenity BDD with Screenplay

It is a framework and open-source library for the creation of automated software testing for code in development. Like other automated testing tools, Serenity BDD is made to smooth development by catching bugs sooner and helping ensure the release of a stable, fully functioning end software product.

Screenplay, introduced in Serenity BDD, aims to encourage good testing habits and well-designed test suites by enabling teams to write more robust and reliable tests. It is based on the Selenium WebDriver and the Page Objects model.

Cucumber

It is a software tool that supports behavior-driven development. Central to the Cucumber BDD approach is its ordinary language parser called Gherkin. It allows expected software behaviors to be specified in a logical language that customers can understand.

Core Java

It is a part of the Java programming language that one can use for developing or creating a general-purpose app. Its focus is to build such general applications. The J2SE (Java Standard Edition) is known as Core Java. It mainly covers concepts of object-oriented programming (OOP).

Gradle

It is a build automation tool for multi-language software development. It controls the development process in the tasks of compilation and packaging to testing, deployment, and publishing. Supported languages include Java, C/C++, and JavaScript.

Maven

It is a build automation tool used primarily for Java projects. Maven can also be used to build and manage projects written in C#, Ruby, Scala, and other languages. The Maven project is hosted by the Apache Software Foundation, where it was formerly part of the Jakarta Project.

Git Hub Actions

It is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository or deploy merged pull requests to production.

JavaScript

It is a programming language that is one of the core technologies of the World Wide Web, alongside HTML and CSS. Over 97% of websites use JavaScript on the client side for web page behavior, often incorporating third-party libraries.

NodeJS

It is an open-source, cross-platform, back-end JavaScript runtime environment that runs on the V8 engine and executes JavaScript code outside a web browser.

Docker

It is a set of platforms as a service product that use OS-level virtualization to deliver software in packages called containers. The service has both free and premium tiers. The software that hosts the containers is called Docker Engine.

AWS

It is a subsidiary of Amazon that provides on-demand cloud computing platforms and APIs to individuals, companies, and governments, on a metered pay-as-you-go basis. These cloud computing web services provide distributed computing processing capacity and software tools via AWS server farms.